

TUTORIEL · CONFIGURATION & BONNES PRATIQUES

Le Code du Claudeur

VOLUME 1

Step-by-step pour configurer Claude et travailler avec lui comme un pro. Copy-paste ready, agnostique de l'interface.

PUBLIC	Devs solo, équipes, débutants à intermédiaires
FORMAT	8 chapitres · 30 min pour démarrer · annexes copy-paste
COUVRE	Claude Code CLI · Web · VS Code · JetBrains · Cursor · Claude.ai
VERSION	V1 · 20 mai 2026

Avant-propos

Ce guide existe parce qu'un Claude bien configuré n'a rien à voir avec un Claude par défaut. Trois heures de setup t'épargnent six mois de frustration.

Pour qui

- **Le solo dev** qui code avec Claude tous les jours et qui veut arrêter de répéter les mêmes prompts.
- **L'équipe** qui veut un Claude qui se comporte de la même façon chez tout le monde.
- **Le débutant qui sort du "vibe coding"** et veut un workflow reproductible.

Ce que ce guide n'est PAS

- Un cours de *prompt engineering* (rédaction de prompt pur).
- Une doc de l'API Claude (pas de code Python avec `anthropic.messages.create()`).
- Un cours de programmation. Tu es supposé savoir lancer un `git commit` et éditer un fichier.

Ce que tu vas apprendre

- Configurer le **contexte permanent** que Claude lit à chaque session (`CLAUDE.md`).
- Automatiser les **actions répétitives** via les hooks (zéro token, déterministe).
- Créer des **rôles spécialisés** (sub-agents) qui font une seule chose, bien.
- Écrire des **slash commands** custom qui wrappent tes workflows en 1 ligne.
- Mettre en place un **Coach mode** qui te rappelle les bons réflexes au bon moment.
- Maintenir tout ça dans le temps, sans dette de config.

Conventions de lecture

- 📄 **Bloc copy-paste** : tout ce qui est dans un encadré avec un trait vert à gauche est prêt à coller tel quel (en adaptant les chemins/noms à ton projet).
- 💡 **Astuce** : un raccourci ou un truc qu'on apprend en pratiquant.
- ⚠️ **Attention** : un piège qui peut te coûter du temps.
- 🚫 **Anti-pattern** : un truc qu'on voit faire mais qui finit toujours mal.

COMMENT LIRE CE GUIDE

Lis les chapitres 0 à 2 d'une traite ($\approx$ 20 min), tu auras déjà un Claude transformé. Les chapitres 3 à 7 peuvent être étalés sur 4 semaines au rythme d'une par semaine, comme dans la prog d'origine. Les annexes sont à garder ouvertes pendant le setup pour le copy-paste.

Sommaire

00	Choisir son interface (CLI, Web, IDE, mobile)	4
01	Philosophie — configurer plutôt que vibe-coder	5
02	Le minimum vital — jour 1, 30 minutes	6
03	Les hooks — semaine 1	9
04	Les sub-agents — semaine 2	12
05	Les slash commands — semaine 3	15
06	Le Coach mode — semaine 4	17
07	Maintenance, mémoire, MCP — au-delà	20
08	Annexes — templates copy-paste	22

00 Choisir son interface

Avant de configurer quoi que ce soit, sache où tu vas l'installer. Toutes les interfaces Claude ne supportent pas les mêmes mécanismes.

Les 6 surfaces officielles Claude Code

Toutes lisent la même config `.claude/` dans ton repo. Tu peux switcher entre elles sans rien re-configurer.

SURFACE	PARTICULARITÉ
Terminal CLI	Le plus complet. Commande <code>claude</code> dans n'importe quel dossier. Idéal pour scripting, pipes, CI.
VS Code extension	Inline diffs, @-mentions, plan review. Marche aussi dans Cursor (extension officielle Anthropic, pas <code>.cursorrules</code>).
JetBrains plugin	IntelliJ, PyCharm, WebStorm, etc. Marketplace JetBrains officielle.
Desktop app	App standalone macOS/Windows. Diff visuel, sessions parallèles, tâches planifiées, sessions cloud.
Web	<code>claude.ai/code</code> dans le navigateur. Idéal pour tâches longues, repos pas en local, mobile.
iOS app	Claude app pour iPhone. Continuer une session depuis le téléphone.

💡 LE MOTEUR EST PARTAGÉ

Chaque surface se connecte au même moteur Claude Code. Ton `CLAUDE.md`, tes hooks, tes sub-agents, ton `.mcp.json` — tout marche partout. Tu démarres une tâche dans le terminal et tu peux la reprendre sur ton tel via la session.

Recommandation par profil

Solo dev qui code dans un IDE

→ **VS Code ou JetBrains extension** pour le quotidien, **CLI** dans le terminal pour les workflows scriptés. Setup complet possible.

Équipe

→ N'importe quelle surface, avec un setup `.claude/` versionné dans git. Tout le monde hérite de la config.

Mobile / hors du bureau

→ **Web** sur tel ou **iOS app**. Lance une tâche longue, reprends-la sur l'ordi avec `claude --teleport`.

Utilisateur Cursor

→ Installer l'extension Claude Code officielle dans Cursor
(`cursor:extension/anthropic.claude-code`). Toute la config `.claude/` marche.

⚠ **CLAUDE.AI (CHAT) ≠ CLAUDE CODE**

Claude.ai (le produit chat) supporte les "Projects" avec instructions custom (équivalent léger à CLAUDE.md), mais **pas** hooks, sub-agents, ou MCP serveur. Ce guide concerne Claude Code (les 6 surfaces ci-dessus). Pour le chat seul, garde uniquement les chapitres 1-2 (principes + CLAUDE.md).

01 Philosophie — configurer plutôt que vibe-coder

La différence entre un dev qui utilise Claude et un dev qui en tire 10× plus, ce n'est pas le prompt. C'est ce qui se passe *autour* du prompt.

Le problème du vibe-coding

Quand tu utilises Claude sans configuration, voici ce qui se passe à chaque session :

1. Tu réexpliques ta stack ("on est en Next.js 16, attention au breaking de revalidate...").
2. Tu réexpliques tes conventions ("camelCase pour les fonctions, kebab-case pour les fichiers...").
3. Claude part dans une direction, tu corriges ("non, ici on utilise pas alert(), c'est un toast Sonner").
4. Claude lance une commande dangereuse, tu valides 4 pop-ups pour des trucs anodins.
5. Tu oublies de lancer `pnpm verify` avant un push, Vercel crash.
6. Tu refais la même séquence demain.

Multiplie par 250 sessions/an. Tu réalises que tu as passé des semaines à faire du **contexte jetable**.

L'idée centrale

Tout ce que tu dis à Claude plus de 3 fois doit vivre dans un fichier. Tout ce que Claude fait plus de 3 fois sans intervention humaine doit être un hook.

Les 3 principes du Claudeur

PRINCIPE 1 — CAPITALISER LE CONTEXTE

Stack, conventions, gotchas, structure du projet → `CLAUDE.md`. Claude le lit à chaque session, automatiquement. Tu n'as plus à expliquer "on est en Next.js 16".

PRINCIPE 2 — DÉTERMINISER L'AUTOMATISABLE

Auto-format après chaque édition, log des commandes, rappels en début de session → **hooks**. Ce sont des scripts (bash, python) qui tournent sur des événements. Zéro token, zéro friction.

PRINCIPE 3 — SPÉCIALISER LES RÔLES

Audit sécu, validation de formules, scaffold de composant → **sub-agents**. Chaque sub-agent a son contexte, son modèle, ses outils. Tu lances un audit sécu sans polluer la session principale.

Le ratio temps investi / temps gagné

SETUP	TEMPS DE MISE EN PLACE	GAIN PAR SESSION	BREAK-EVEN
CLAUDE.md de base	15 min	2-5 min	3 sessions
Permissions allow	10 min	1-3 pop-ups évités	1 session
Hook PreToolUse défensif	20 min	0 (mais sauve 1 catastrophe/an)	1 ^{re} catastrophe évitée
Hook PostToolUse format	15 min	30 secondes / Write	3 sessions
5 sub-agents ciblés	2 h	10-30 min / audit	~10 audits
3 slash commands	45 min	2 min / commande	~20 invocations

Total setup : **une demi-journée**. Gain estimé : **15-30 min par jour de dev actif**. Sur 200 jours/an, ça fait 50-100 heures économisées.

02 Le minimum vital — jour 1

Objectif de ce chapitre : transformer ton Claude en 30 minutes. Aucun pré-requis. Juste 3 fichiers à créer.

Étape par étape

1

Installer Claude Code

Selon ton OS et ta surface :

```
# macOS, Linux, WSL – installation native (recommandée, auto-update)
curl -fsSL https://claude.ai/install.sh | bash

# Windows PowerShell
irm https://claude.ai/install.ps1 | iex

# macOS via Homebrew
brew install --cask claude-code

# Windows via WinGet
winget install Anthropic.ClaudeCode
```

BASH

Puis dans n'importe quel dossier de projet :

```
cd ton-projet
claude
```

BASH

Pour les IDE : Extensions → chercher "Claude Code" (VS Code, Cursor, JetBrains). Pour le navigateur : `claude.ai/code` . Pour l'app : télécharger depuis `claude.ai`. Toutes ces surfaces lisent les mêmes fichiers `.claude/` .

Créer `CLAUDE.md` à la racine du projet

C'est le brain de ton projet. Claude le charge automatiquement à chaque session.
Template minimal :

```
# <Nom du projet>

Description en 1 phrase de ce que fait l'app.

## Stack technique
- Langage : <Python 3.12 | TypeScript 5 | Go 1.22 | ...>
- Framework : <Next.js 16 | FastAPI | Rails | ...>
- DB : <Postgres via Supabase | SQLite | MongoDB | ...>
- Tests : <pytest | vitest | jest>
- Déploiement : <Vercel | Fly.io | Railway | ...>

## Commandes utiles
- `` – lancer le serveur local
- `` – type-check avant push (ce qui bloque le déploiement)
- `` – lancer la suite de tests

## Conventions
- <Une règle de nommage> (ex: camelCase JS, kebab-case fichiers)
- <Une règle de structure> (ex: chaque page a son `actions.ts`)
- <Une règle de comportement> (ex: jamais `alert()`, toujours toast Sonner)

## Gotchas
- <Piège connu de la stack> (ex: Next.js 16 a changé `revalidate`)
- <Piège interne au projet>

## Workflow Git
- Branche cible : `<master | main>`
- Format de commit : `type(scope): description`
- <PR ou push direct>

## Règles impératives
- **Ne jamais** <X> sans confirmation
- **Toujours** <Y> avant <Z>
```

CLAUDE.MD

3

Créer `.claude/settings.json`

Le fichier qui contrôle les permissions et les hooks. Template minimal qui élimine 80 % des pop-ups :

```
.CLAUDE/SETTINGS.JSON
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "permissions": {
    "allow": [
      "Read",
      "Grep",
      "Glob",
      "Bash(git status:*)",
      "Bash(git diff:*)",
      "Bash(git log:*)",
      "Bash(git show:*)",
      "Bash(git branch:*)",
      "Bash(ls:*)",
      "Bash(cat:*)",
      "Bash(find:*)",
      "Bash(head:*)",
      "Bash(tail:*)",
      "Bash(wc:*)"
    ],
    "deny": [
      "Bash(rm -rf*)",
      "Bash(sudo*)",
      "Bash(curl* | bash*)"
    ]
  }
}
```

Adapte la liste `allow` à ta stack (ajoute `"Bash(pnpm verify)"`, `"Bash(npm test:*)"`, etc.).

4

Ajouter `.claude/` **au** `.gitignore` **(sélectivement)**

Tu veux versionner les fichiers d'équipe mais pas les logs locaux. Ajoute à ton `.gitignore` :

```
# Claude Code – versionner agents/commands/hooks/settings.json
# mais pas les logs ni les overrides perso
.claude/*
!.claude/agents/
!.claude/commands/
!.claude/hooks/
!.claude/settings.json
.claude/logs/
.claude/settings.local.json
.GITIGNORE
```

Lance Claude Code. Pose une question : *"Lis CLAUDE.md et résume la stack en 3 lignes."* S'il te répond correctement avec ce que tu as mis dans le fichier →  ça marche. Sinon : vérifie que le fichier est bien à la racine du projet, pas dans `src/` ou ailleurs.

INITIALISATION AUTO

Claude Code propose souvent une commande `/init` qui scanne ton repo et propose une première version de CLAUDE.md. Lance-la, puis ajuste : retire ce qui est faux, ajoute tes règles spécifiques.

ANTI-PATTERN

Mettre 800 lignes dans CLAUDE.md le premier jour. Le fichier doit rester **stable et lisible**. Si tu te retrouves à scroller pour trouver une info, il est temps de splitter (cf. chapitre 7).

Ce que tu as gagné en 30 minutes

- Plus jamais besoin de réexpliquer la stack ou les conventions.
- Plus de pop-ups pour les commandes git/ls/cat classiques.
- Les commandes dangereuses (`rm -rf` , `sudo`) sont bloquées.
- Le repo est prêt pour les niveaux suivants.

03 Les hooks — semaine 1

Un hook est un script qui se déclenche sur un événement. Zéro token, zéro intervention de Claude, zéro friction. C'est la couche "automation déterministe" que Claude ne voit même pas.

Les 4 événements clés (parmi ~25)

Claude Code expose environ 25 événements (UserPromptSubmit, SessionEnd, SubagentStart/Stop, FileChanged, PreCompact, Notification, etc.). Ce guide couvre les **4 qui couvrent 80 % des besoins**. La liste complète est dans la doc officielle

code.claude.com/docs/en/hooks.

ÉVÉNEMENT	QUAND	CAS TYPIQUES
SessionStart	Au démarrage de Claude	Récap git, fetch silencieux, état du projet
PreToolUse	Avant chaque appel d'outil	Bloquer <code>rm -rf</code> , écriture sur <code>.env</code> , push <code>--force</code>
PostToolUse	Après chaque appel d'outil	Auto-format, log d'activité
Stop	Quand Claude finit son tour	Notif système, Coach mode, ping Slack/Discord

Hook #1 — PostToolUse : auto-format après chaque écriture

Crée le fichier `.claude/hooks/post-edit-format.sh` :

```
#!/usr/bin/env bash
# Auto-format les fichiers TS/JS après Write/Edit via ESLint --fix.
# Adapter le formater selon ta stack : prettier, black, gofmt, rustfmt...
set -e

INPUT="$(cat 2>/dev/null || echo '{}')"
FILE=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty' 2>/dev/null)
[ -z "$FILE" ] || [ ! -f "$FILE" ] && exit 0

case "$FILE" in
  *.ts|*.tsx|*.js|*.jsx|*.mjs) ;;
  *) exit 0 ;;
esac

# Ignorer node_modules, build, migrations
case "$FILE" in
```

BASH

```

    *node_modules*|*.next/*|*build/*|*dist/*) exit 0 ;;
esac

cd "$(dirname "$0")/../../.."
pnpm exec eslint --fix "$FILE" >/dev/null 2>&1 || true
exit 0

```

N'oublie pas `chmod +x .claude/hooks/post-edit-format.sh`.

Hook #2 — PreToolUse : garde-fou défensif

Ce hook est **le plus important**. Il bloque les commandes destructrices avant qu'elles s'exécutent. Crée `.claude/hooks/pre-tool-guard.sh` :

```

#!/usr/bin/env bash BASH
# Garde-fous : bloque les actions destructrices avant exécution.

INPUT="$(cat 2>/dev/null || echo '{}')"
TOOL=$(echo "$INPUT" | jq -r '.tool_name // empty' 2>/dev/null)

# 1. Bloque toute écriture sur un .env*
if [ "$TOOL" = "Edit" ] || [ "$TOOL" = "Write" ]; then
    FILE=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty' 2>/dev/null)
    case "$FILE" in
        *.env|*.env.*|.env)
            echo "BLOCKED: fichier .env protégé. Utilise les env vars du provider." >&2
            exit 2
        ;;
    esac
fi

# 2. Bloque les Bash dangereux
if [ "$TOOL" = "Bash" ]; then
    CMD=$(echo "$INPUT" | jq -r '.tool_input.command // empty' 2>/dev/null)

    # rm -rf sur chemins dangereux
    if echo "$CMD" | grep -qE 'rm[[:space:]]+-[a-zA-Z]*[rf][a-zA-Z]*[[:space:]]+(/|~|\.|\.|/|HOME)'; then
        echo "BLOCKED: rm -rf sur chemin dangereux." >&2
        exit 2
    fi

    # git push --force
    if echo "$CMD" | grep -qE 'git[[:space:]]+push.*(--force|--force-with-lease|[[:space:]]+-[[:space:]]+)$'; then
        echo "BLOCKED: git push --force. Demande confirmation explicite à l'user." >&2
        exit 2
    fi

    # git commit --no-verify (skip des pre-commit hooks)
    if echo "$CMD" | grep -qE 'git[[:space:]]+commit.*--no-verify'; then
        echo "BLOCKED: --no-verify skip les checks. Fix les erreurs d'abord." >&2
        exit 2
    fi
fi

```

```
exit 0
```

Hook #3 — SessionStart : récap projet

À l'ouverture d'une session, affiche l'état du projet en 5 lignes. Crée `.claude/hooks/session-start.sh` :

```
#!/usr/bin/env bash
# Récap à l'ouverture : branche, dirty files, commits ahead/behind, 3 derniers commits.
set -e
cd "$(dirname "$0")/../../.."

git fetch origin --quiet 2>/dev/null || true

BRANCH=$(git rev-parse --abbrev-ref HEAD 2>/dev/null || echo "?")
DIRTY=$(git status --porcelain 2>/dev/null | wc -l | xargs)
AHEAD=$(git rev-list --count @{u}..HEAD 2>/dev/null || echo "?")
BEHIND=$(git rev-list --count HEAD..@{u} 2>/dev/null || echo "?")

echo ""
echo "===== "
echo "  <Projet> · session démarrée · $(date '+%a %d %b %H%M') "
echo "===== "
echo "  Branch      : $BRANCH"
[ "$DIRTY" = "0" ] && echo "  Working tree : clean" || echo "  Working tree : $DIRTY fichier(s) modifié(s)"
[ "$AHEAD" != "0" ] && [ "$AHEAD" != "?" ] && echo "  ▲ $AHEAD commit(s) non pushé(s)"
[ "$BEHIND" != "0" ] && [ "$BEHIND" != "?" ] && echo "  ▲ $BEHIND commit(s) en retard sur origin"
echo ""
echo "  Derniers commits :"
git log --oneline -3 2>/dev/null | sed 's/^/    /'
echo "===== "
echo ""
```

BASH

Activer les hooks dans `settings.json`

Ajoute la section `"hooks"` à ton `.claude/settings.json` :

```
{
  "permissions": { /* ... cf. chapitre 2 ... */ },
  "hooks": {
    "SessionStart": [
      { "matcher": "*", "hooks": [
        { "type": "command", "command": ".claude/hooks/session-start.sh" }
      ] }
    ],
    "PreToolUse": [
      { "matcher": "Edit|Write|Bash", "hooks": [
        { "type": "command", "command": ".claude/hooks/pre-tool-guard.sh" }
      ] }
    ],
    "PostToolUse": [
```

.CLAUDE/SETTINGS.JSON

```

    { "matcher": "Edit|Write", "hooks": [
      { "type": "command", "command": ".claude/hooks/post-edit-format.sh" }
    ]
  }
}

```

⚠️ PRÉ-REQUIS

Les hooks bash ci-dessus utilisent `jq` pour parser le JSON du payload. Installe-le si pas déjà présent : `brew install jq` (macOS) ou `apt install jq` (Linux). Sinon, écris tes hooks en Python — équivalent en 5 lignes avec `json.load(sys.stdin)`.

Tester un hook sans redémarrer la session

```

# Simuler un événement PreToolUse
echo '{"tool_name":"Bash","tool_input":{"command":"rm -rf /"}}' \
  | .claude/hooks/pre-tool-guard.sh
echo "Exit code: $?"
# Doit afficher "BLOCKED: rm -rf..." et exit 2

```

BASH

💡 PERFORMANCE

Un hook doit s'exécuter en **moins de 200 ms**. Au-delà, tu sentiras de la friction. Si ton hook fait des appels réseau ou lance un linter lourd, marque-le `"async": true` dans la config — il tourne en arrière-plan sans bloquer.

04 Les sub-agents — semaine 2

Un sub-agent est un rôle spécialisé avec son propre contexte, son modèle, ses outils, et sa system prompt. Tu le déclenches pour une tâche précise et il revient avec un résultat structuré.

Pourquoi diviser plutôt que tout faire en un

1. **Contexte préservé** — chaque agent démarre avec un contexte propre, ne pollue pas la session principale.
2. **Périmètre étroit** — un agent d'audit n'a pas les outils Write/Edit. Il ne peut *pas* casser du code.
3. **Modèle adapté** — un audit complexe utilise `opus`, une classification rapide utilise `haiku`. Tu optimises le coût.
4. **Spécialisation** — l'agent de sécurité connaît OWASP ; l'agent métier connaît les conventions de ton domaine. Pas mélangé.

Anatomie d'un agent

Un agent est un fichier markdown dans `.claude/agents/<nom>.md` avec un frontmatter YAML et une system prompt en markdown.

```
---
name: code-auditor
description: >
  Audit READ-ONLY de qualité de code. Use proactively quand l'utilisateur
  dit "audite le code", "passe en revue", ou avant un gros refactor.
  Détecte : code mort, duplication, complexité, anti-patterns, types `any`
  injustifiés. Ne modifie AUCUN fichier – produit un rapport structuré.
model: opus
tools: Read, Grep, Glob, Bash, WebFetch
---

Tu es un auditeur de code senior pour le projet <nom>. Tu travailles
uniquement en lecture.

## Contexte projet
- Stack : <rappel court>
- Conventions à respecter (cf. CLAUDE.md) : <rappel>

## Ta mission
Pour chaque fichier audité, identifie :
1. Code mort : fonctions, variables, imports inutilisés
2. Duplication : patterns répétés qui devraient être factorisés
3. Complexité : fonctions > 30 lignes, conditionnelles imbriquées > 3 niveaux
```

4. **Anti-patterns** : useEffect mal utilisés, requêtes non parallélisées, types ``any``

Format de sortie

Pour CHAQUE finding :

- **Sévérité** : ● / ● / ● / ●
- **Fichier:ligne**
- **Constat** + **Impact** + **Fix recommandé**

Termine par : Score global /10 + Top 3 actions prioritaires.

Règles

- Lecture seule : pas d'Edit, pas de Write.
- Ne nitpicker pas le style si déjà couvert par le linter.
- Prioriser ce qui a un vrai impact sur la maintenabilité.

Choisir le modèle

Le champ `model` du frontmatter accepte un alias (`sonnet` , `opus` , `haiku`), un ID complet (ex: `claude-opus-4-7`), ou `inherit` (par défaut). Le tarif exact varie ; en pratique, sonnet et opus coûtent plusieurs fois plus que haiku.

MODÈLE	QUAND
haiku	Classification, triage rapide, exploration codebase, tâches répétitives
sonnet	Génération de code, audits standards, contenu structuré — le défaut sain
opus	Audits complexes (sécu, DB, archi), raisonnement long, décisions stratégiques

Règle de pouce : tout audit qui peut faire perdre des données ou compromettre la prod → opus. Tout le reste → sonnet par défaut. Haiku quand la rapidité prime sur la profondeur.

Le champ `effort` contrôle l'effort de raisonnement (séparé du modèle). Valeurs : `low` , `medium` , `high` , `xhigh` , `max` — les niveaux disponibles dépendent du modèle.

Restreindre les outils (moindre privilège)

Donne uniquement les outils dont l'agent a besoin. Un auditeur read-only ne reçoit jamais Write/Edit/Bash dangereux.

Agent d'audit (read-only)

tools: Read, Grep, Glob, Bash,
WebFetch, WebSearch
Pas de Write/Edit.

Agent qui crée du code

tools: Read, Write, Edit, Bash, Glob
Bash uniquement si nécessaire (tests).

Agent qui écrit du contenu

tools: Read, Write, Edit, Grep,
WebFetch, WebSearch
Pas de Bash.

Agent de classification

tools: Read, Grep, Glob
Minimaliste, modèle haiku.

La description "Use proactively when..."

Claude lit la `description` de chaque agent et décide automatiquement quand l'invoquer. Plus la description est précise, plus l'invocation est juste.

```
description: >
  <Ce que fait l'agent en 1 phrase>.
  Use proactively when user says "<mot-clé 1>", "<mot-clé 2>", "<mot-clé 3>",
  or before <événement déclencheur>.
  Covers: <liste de ce qui est couvert>.
  <"Read-only" ou "Modifies files in X">.
```

FRONTMATTER

Combien d'agents au maximum

⚠ L'USINE À AGENTS

Au-delà de **10-12 agents**, tu ne te souviens plus de ce que chacun fait. Tu finis par utiliser toujours les 3 mêmes. Garde un dossier propre : 5-8 agents bien ciblés > 20 agents oubliés.

Tu peux commencer avec ces 3 agents qui couvrent 80 % des cas :

1. `code-auditor` — audit qualité (template au-dessus)
2. `security-auditor` — RLS, secrets, OWASP, RGPD (cf. annexe F)
3. `qa-tester` — flows utilisateur de bout en bout après gros chantier

Ajoute les autres au fur et à mesure que tu identifies un besoin *récurrent*.

Invoquer un sub-agent

- **Automatique** — Claude lit la `description` et déclenche l'agent quand le pattern match.
- **Explicite** — l'utilisateur tape *"Lance security-auditor sur les server actions de la semaine"*.
- **Via un slash command** — qui wrap l'invocation avec du contexte pré-formaté (cf. chapitre 5).

TESTER UN AGENT

Après création, invoque-le une fois sur un cas réel. Lis le rapport. S'il rate des choses évidentes ou si son ton ne te plaît pas : ajuste la system prompt. Itère 2-3 fois avant de considérer l'agent stable.

05 Skills & slash commands —

semaine 3

Un skill (ou slash command) est un raccourci qui te donne accès à un workflow en tapant `/<nom>`. C'est ta couche "playbooks réutilisables".

⚠ FUSION SKILLS + COMMANDS

Depuis fin 2025, les "custom commands" et les "skills" sont **fusionnés** dans Claude Code. Un fichier `.claude/commands/deploy.md` et un skill `.claude/skills/deploy/SKILL.md` créent tous les deux la commande `/deploy` et fonctionnent pareil. Tes anciens fichiers `.claude/commands/` marchent encore (rétrocompat). La voie moderne recommandée est `.claude/skills/` — elle débloque : fichiers de support à côté, contrôle d'invocation, chargement automatique par Claude quand pertinent.

Skill vs sub-agent vs hook

	SKILL / SLASH COMMAND	SUB-AGENT	HOOK
Déclenchement	Tu tapes <code>/nom</code> ou Claude le charge auto	Auto ou explicite	Événement (auto)
Contexte	Session courante	Nouveau contexte isolé	N/A (script)
Tokens	Oui	Oui (overhead nouveau contexte)	Zéro
Cas idéal	Workflow récurrent, doc/checklist	Expertise isolée, tâche lourde	Automation déterministe

Deux façons d'en créer un

Voie A — Simple : `.claude/commands/<nom>.md`

Un seul fichier markdown avec frontmatter + instructions. Convient à 90 % des cas.

```
---
description: "Verify + commit + push en 1 commande"
argument-hint: "<message de commit>"
.CLAUDE/COMMANDS/SHIP.MD
```

```

---

# /ship – Verify, commit & push

## Procédure

### 1. Vérification préalable
Lance la commande de typecheck (ex: `pnpm verify`). Si rouge → STOP, affiche
les erreurs, demande à l'utilisateur s'il veut fixer ou abandonner. Ne commit
jamais avec un typecheck rouge.

### 2. État Git
Lance `git status --short` et `git diff --stat`. Affiche les fichiers modifiés
et un résumé du diff. Si zéro fichier modifié → "Rien à commiter" et stop.

### 3. Construction du commit
Le message = `$ARGUMENTS`. Si vide ou < 10 caractères, demande un message
plus précis. Formate au style projet : `type(scope): summary` (cf. CLAUDE.md).

### 4. Stage + commit + push
Exécute dans cet ordre :
1. `git add` des fichiers concernés (pas `git add .` aveugle)
2. `git commit -m "...` avec le message construit
3. `git push origin <branche cible>`

### 5. Confirmation
Affiche le SHA, le message, le récap insertions/deletions.

## Garde-fous
- Ne jamais utiliser `--no-verify` sauf demande explicite.
- Ne jamais force-push sans confirmation explicite.
- Si le push échoue (rejected) : pull `--no-rebase`, résous conflits, re-push.

```

Voie B — Puissante : `.claude/skills/<nom>/SKILL.md`

Un dossier dédié avec un `SKILL.md` + fichiers de support (scripts, références, exemples).
Convient quand le workflow utilise des scripts bundlés ou des docs détaillées.

```

.claude/skills/deploy/
├─ SKILL.md           ← Instructions principales (requis)
├─ reference.md       ← Doc détaillée (chargée à la demande)
├─ scripts/
│   └─ verify.sh      ← Script exécutable

```

STRUCTURE

```

---
name: deploy
description: Deploy to production with verify + commit + push
disable-model-invocation: true
allowed-tools: Bash(pnpm verify) Bash(git *)
---

# /deploy – Production deploy

Use the script bundled with this skill:

```bash
bash ${CLAUDE_SKILL_DIR}/scripts/verify.sh

```

.CLAUDE/SKILLS/DEPLOY/SKILL.MD

```

```

Then commit and push following the standard workflow...

Frontmatter clés des skills (tous optionnels sauf `description`) :

- `description` — quand Claude doit utiliser le skill (recommandé)
- `disable-model-invocation: true` — seul l'utilisateur peut invoquer (recommandé pour `/deploy`, `/ship`, etc. qui ont des effets de bord)
- `allowed-tools` — outils pré-autorisés quand le skill tourne
- `argument-hint` — aide auto-complétion (ex: `<message de commit>`)
- `context: fork` — exécuter dans un sub-agent isolé
- `paths` — glob patterns qui limitent quand le skill est activé auto

## 3 commands à coder en semaine 3

### `/ship "<message>"` — déployer en 1 ligne

Template ci-dessus. C'est **le command le plus utile**. Il remplace ta routine commit/push manuelle et garantit que le typecheck passe.

### `/audit-quick` — audit code + sécu en parallèle

```

description: "Audit éclair : code + sécu en parallèle, rapport synthétique"

/audit-quick – Audit avant chantier important

Lance ces 2 agents EN PARALLÈLE (même message, plusieurs tool calls) :

Agent code-auditor :
> Audit code Pivola, focus : code mort, duplication, anti-patterns,
> `any` injustifiés. Rapport ≤ 500 mots hiérarchisé 🚫/🔴/🟡.

Agent security-auditor :
> Audit sécu Pivola, focus : RLS, secrets, XSS, exposition PII, OWASP.
> Rapport ≤ 500 mots hiérarchisé 🚫/🔴/🟡.

Quand les 2 ont fini, présente un rapport unifié :
- 🚫 Bloquants
- 🔴 Important
- 🟢 OK

Termine par une seule reco prioritaire.
```

## /standup — récap quotidien

.CLAUDE/COMMANDS/STANDUP.MD

```

description: "Récap du matin : commits 24h, WIP, focus suggéré"

/standup

Lance en parallèle :
- `git log --since='24 hours ago' --oneline`
- `git status --short` + `git diff --stat`

Rapport en 3 sections, ≤ 200 mots :
- 🟢 Fait hier
- 🟡 En cours (WIP)
- 🎯 Focus du jour (suggestion)

Termine par une question : "Tu attaques le focus #1 ou autre chose ?"
```

## Le pattern \$ARGUMENTS

Quand l'utilisateur tape `/ship "fix: typo dans header"`, la string entre guillemets est disponible dans la slash command via `$ARGUMENTS`. Utilise-le pour personnaliser le comportement.

### 💡 DISAMBIGUATION

Crée un fichier `.claude/COMMANDS.md` (hors du dossier `commands/` sinon ça créerait une slash command `/COMMANDS`) qui liste toutes tes commandes avec un tableau "Tu veux X → commande Y". Sauve du temps quand tu as 10+ commandes.

### 🚫 ANTI-PATTERN

Mettre un `README.md` dans `.claude/commands/`. Claude Code charge **tous** les `.md` de ce dossier comme des slash commands. Tu te retrouverais avec une commande `/README`. Range les docs hors du dossier.



## 06 Le Coach mode — semaine 4

Coach mode = un système qui te rappelle les bons réflexes au bon moment, sans que tu doives y penser. Trois mécanismes complémentaires : règles dans CLAUDE.md, hook Stop qui observe, slash `/coach` manuel.

### Pourquoi un Coach

Une fois ton setup en place, tu accumules des slash commands utiles ( `/ship` , `/audit-quick` , `/loro` , etc.). Mais dans le feu de l'action, tu oublies de les utiliser. Tu finis par taper `git commit + git push` à la main, sans verify, et Vercel crash.

Le Coach répare ça en 3 couches.

### Couche 1 — Règles dans CLAUDE.md

Ajoute une section "*Coach mode*" dans `CLAUDE.md` avec une table de triggers. Je la relis à chaque session.

```
Coach mode – quand proposer quoi
```

CLAUDE.MD

```
Triggers à honorer avant d'agir. Si une condition est vraie,
proposer la commande/agent au lieu de foncer.
```

```
| Trigger observé | Action à proposer |
|---|---|
| Tâche touche ≥ 5 fichiers ou nouvelle feature transversale | `/new-feature <slug>` |
| Modification d'un widget UI | `/loro <file>` avant ship |
| Modification d'un calcul critique | `/trading-check <fn>` après la modif |
| Migration SQL | agent `db-schema-reviewer` avant push |
| Server action / route auth modifiée | agent `security-auditor` avant push |
| ≥ 10 fichiers modifiés depuis dernier commit | `/audit-quick` avant ship |
| Avant tout `git push` | `/ship "<message>"` |
```

```
Règle générale : si une commande couvre ~80 % de ce qui allait
être fait à la main, proposer avant d'agir. L'utilisateur peut
toujours répondre "non, fais à la main" – c'est ok.
```

## Couche 2 — Hook Stop qui observe

Le hook `Stop` se déclenche quand Claude finit son tour. Il lit le log d'activité de la session courante et imprime des suggestions ciblées **dans ton terminal**, sans consommer de tokens.

Pré-requis : avoir un hook `PostToolUse` qui log les `Write/Edit/Bash` dans

`.claude/logs/activity.log` (cf. annexe D). Puis crée `.claude/hooks/coach-suggest.sh` :

```
#!/usr/bin/env bash
Hook Stop – Coach. Analyse l'activity-log de la session courante
et imprime des suggestions selon ce qui a été touché.
set -e
cd "$(dirname "$0")/../../.."

Coach mute : si le flag existe, on sort silencieusement
[-f ".claude/coach-mute"] && exit 0

LOG=".claude/logs/activity.log"
[! -f "$LOG"] && exit 0

INPUT="$(cat 2>/dev/null || echo '{}')"
SID=$(echo "$INPUT" | jq -r '.session_id // empty' | cut -c1-8)
[-z "$SID"] && exit 0

RECENT=$(grep "\[$SID\]" "$LOG" 2>/dev/null | tail -200)
[-z "$RECENT"] && exit 0

FILES=$(echo "$RECENT" | grep -E 'WRITE|EDIT' | sed 's/.*| //' | sort -u)
WRITES=$(echo "$RECENT" | grep -cE 'WRITE|EDIT')
["$WRITES" -eq 0] && exit 0

SUGGESTIONS=""

Trigger 1 : widget touché → /loro
if echo "$FILES" | grep -q "components/*.widget"; then
 W=$(echo "$FILES" | grep "components/*.widget" | head -1)
 SUGGESTIONS="${SUGGESTIONS} • /loro $W\n → polish du widget modifié\n"
fi

Trigger 2 : auth / API touché → security-auditor
if echo "$FILES" | grep -qE "actions\.ts|/auth|/api/"; then
 SUGGESTIONS="${SUGGESTIONS} • Agent security-auditor\n → check RLS / secrets sur routes touchées\n"
fi

Trigger 3 : ≥ 2 écritures → /ship
["$WRITES" -ge 2] && SUGGESTIONS="${SUGGESTIONS} • /ship \"<message>\"\n → quand prêt à pousser\n"

[-z "$SUGGESTIONS"] && exit 0

{
 echo ""
 echo "_____ "
 echo " Coach · suggestions"
 echo "_____ "
 printf "%b" "$SUGGESTIONS"
}
```

BASH

```
 echo ""
} >&2

exit 0
```

Active-le dans `settings.json` :

```
"Stop": [
 { "matcher": "*", "hooks": [
 { "type": "command", "command": ".claude/hooks/coach-suggest.sh" }
]}
]
```

.CLAUDE/SETTINGS.JSON

## Couche 3 — Slash `/coach` manuel

Pour les moments où tu hésites toi-même sur quoi faire ensuite. Crée

`.claude/commands/coach.md` :

```

description: "Analyse l'état du repo et recommande la prochaine action"

/coach

Lance en parallèle :
- `git status --short`, `git diff --stat`, `git log --oneline -5`
- Lis `.claude/logs/activity.log | tail -50` si présent

Compare l'état aux triggers Coach dans CLAUDE.md. Recommande max 3
actions priorisées :

1. <commande> → <raison en 1 phrase>
2. <commande> → <raison>
3. <commande> → <raison>

Termine : "Tu veux que je lance la #1 ou tu préfères réfléchir ?"
```

.CLAUDE/COMMANDS/COACH.MD

## Le mécanisme mute

Parfois tu sais ce que tu fais et les suggestions deviennent du bruit. Le hook check un flag

`.claude/coach-mute` et exit silencieusement s'il existe. Deux commandes pour toggle :

- `/coach-mute` → `touch .claude/coach-mute` (Coach OFF)
- `/coach-on` → `rm -f .claude/coach-mute` (Coach ON)

### 💡 ÉVOLUTION NATURELLE

Quand tu refais 3 fois la même action manuellement, c'est le signal pour ajouter un nouveau trigger Coach. Ouvre `CLAUDE.md` → table Coach mode → ajoute une ligne. Si tu veux que le hook le détecte aussi, ajoute la détection dans `coach-suggest.sh`. C'est ce qui transforme une routine consciente en réflexe automatique.

# 07 Maintenance, mémoire, MCP — au-delà

Ton setup vit. CLAUDE.md se périmé, des agents dorment, de nouveaux MCP apparaissent. Voici la maintenance qui le garde sain.

## Mémoire persistante des agents

Par défaut, un sub-agent oublie tout entre deux invocations. Pour les agents critiques (audit sécu, expert métier), tu peux activer une mémoire trans-sessions versionnée en git.

### Pattern

1. Crée le dossier mémoire selon le scope voulu (cf. tableau ci-dessous)
2. Ajoute `memory: project` (ou `user`, ou `local`) au frontmatter de l'agent
3. Claude Code injecte automatiquement les instructions de lecture/écriture dans la system prompt du sub-agent, et active les outils Read/Write/Edit pour qu'il puisse gérer ses fichiers

Trois scopes possibles :

SCOPE	EMPLACEMENT	QUAND
user	<code>~/.claude/agent-memory/&lt;nom&gt;/</code>	Savoir trans-projets (préférences perso, patterns réutilisables)
project	<code>.claude/agent-memory/&lt;nom&gt;/</code>	Savoir projet, versionné en git, partageable en équipe (défaut recommandé)
local	<code>.claude/agent-memory-local/&lt;nom&gt;/</code>	Savoir projet mais sensible / personnel, gitignored

Structure typique d'un dossier mémoire :

```
.claude/agent-memory/security-auditor/
├─ MEMORY.md ← INDEX (1res 200 lignes ou 25KB chargées automatiquement)
├─ feedback_<topic>.md ← Corrections / validations utilisateur
├─ project_<topic>.md ← Décisions, deadlines, motivations
└─ reference_<topic>.md ← Pointeurs vers systèmes externes (Linear, Grafana...)
```

STRUCTURE

Bénéfice typique : un expert métier construit au fil des sessions une base de "formules validées" ou de "conventions du projet" qui survit aux re-clones et aux changements de machine. Plus besoin de re-débattre à chaque session.

## MCP — connecter Claude au monde extérieur

Le **Model Context Protocol** donne à Claude l'accès à des outils externes (DB, browser, filesystem, API). Configuration dans `.mcp.json` à la racine :

```

{
 "mcpServers": {
 "playwright": {
 "command": "npx",
 "args": ["-y", "@playwright/mcp@latest", "--browser=chromium", "--isolated"]
 },
 "filesystem": {
 "command": "npx",
 "args": ["-y", "@modelcontextprotocol/server-filesystem", "./data"]
 },
 "tavily": {
 "type": "http",
 "url": "https://mcp.tavily.com/mcp/",
 "headers": { "X-API-Key": "${TAVILY_API_KEY}" }
 }
 }
}

```

.MCP.JSON

### Quels MCP installer en priorité

MCP	POURQUOI	NIVEAU
Playwright	Tester l'UI dans un browser. "Ouvre le dashboard et vérifie X."	essentiel pour app web
filesystem	Donner accès lecture à un dossier précis (data CSV, exports)	utile cas par cas
Tavily	Recherche web premium (mieux que WebSearch par défaut)	utile si recherche fréquente
DB (Supabase, Postgres...)	Lire le schéma DB sans copier-coller — <b>verrouille en read-only</b>	utile si DB centrale

#### ⚠ SÉCURITÉ MCP

Un MCP qui peut écrire en DB de prod = catastrophe potentielle. Verrouille en **read-only** tant que possible. L'écriture passe par une migration explicite que tu reviews à la main.

## Audit mensuel du setup

Tous les ~30 jours, lance un méta-audit pour détecter les déchets. Crée

`.claude/commands/audit-claude-setup.md` qui check :

- **Agents dormants** : grep `activity.log` pour les agents jamais invoqués depuis création
- **Références périmées** dans CLAUDE.md : fichiers mentionnés qui n'existent plus
- **Commands non documentées** dans COMMANDS.md
- **Permissions** `allow` jamais utilisées (bruit dans settings.json)
- **Mémoires vides** depuis > 30 jours (l'agent ne capitalise rien)
- **Hooks lents** (> 500 ms = friction)

## Splitter CLAUDE.md quand il grossit

Quand CLAUDE.md dépasse ~100 lignes, le splitter en fichiers thématiques importés via `@` :

```
@AGENTS.md
@.claude/STACK.md
@.claude/CONVENTIONS.md
@.claude/WORKFLOW.md

<Projet> – hub de configuration

| Tu cherches... | Va dans |
| ---|---|
| Commandes + stack + gotchas | `@.claude/STACK.md` |
| Conventions code, règles dev, workflow Git | `@.claude/CONVENTIONS.md` |
| Agents, slash commands, Coach mode | `@.claude/WORKFLOW.md` |
| Recettes copy-paste | `@.claude/PATTERNS.md` |
```

CLAUDE.MD

CLAUDE.md devient un hub qui pointe ; les fichiers thématiques restent < 80 lignes chacun, lisibles d'un coup d'œil.

## Partager le setup en équipe

Tout `.claude/` versionné en git = chaque membre hérite automatiquement. Pour aller plus loin, package en plugin :

```
.claude-plugin/<nom>/
├─ plugin.json (métadonnées)
├─ CLAUDE.md (project brain)
├─ settings.json.template (template à customiser)
├─ hooks.json (référence les scripts hooks/)
├─ agents/ (sub-agents)
├─ skills/ (skills)
└─ hooks/ (scripts)
```

STRUCTURE

Le plugin se partage comme une dépendance. Utile quand plusieurs projets internes partagent le même setup de base.



## 08 Annexes — templates copy-paste

Tous les templates de ce guide, regroupés et prêts à coller. Adapte les chemins, noms de projet, et stack à ton contexte.

### A. `CLAUDE.md` universel

# <Nom du projet>

CLAUDE.MD

Description en 1 phrase.

## Stack

- Langage / Framework / DB / Tests / Déploiement

## Commandes utiles

- ``<dev>``, ``<verify>``, ``<test>``, ``<build>``

## Conventions

- Nommage, structure, comportement

## Gotchas

- Pièges connus de la stack et du projet

## Workflow Git

- Branche, format commit, PR/push

## Règles impératives

- Jamais X sans confirmation

- Toujours Y avant Z

## Coach mode (cf. chapitre 6)

| Trigger | Action |

| --- | --- |

| ... | ... |

### B. `.claude/settings.json` complet

```
{
 "$schema": "https://json.schemastore.org/claude-code-settings.json",
 "permissions": {
 "allow": [
 "Read", "Grep", "Glob",
 "Bash(git status:*)", "Bash(git diff:*)", "Bash(git log:*)",
 "Bash(git show:*)", "Bash(git branch:*)", "Bash(git fetch:*)",
 "Bash(ls:*)", "Bash(cat:*)", "Bash(find:*)",
 "Bash(head:*)", "Bash(tail:*)", "Bash(wc:*)"
],
 "deny": [
 "Bash(rm -rf*)", "Bash(rm)", "Bash(sudo*)",
```

.CLAUDE/SETTINGS.JSON

```

 "Bash(curl* | bash*)"
]
},
"hooks": {
 "SessionStart": [
 { "matcher": "*", "hooks": [
 { "type": "command", "command": ".claude/hooks/session-start.sh" }
]}
],
 "PreToolUse": [
 { "matcher": "Edit|Write|Bash", "hooks": [
 { "type": "command", "command": ".claude/hooks/pre-tool-guard.sh" }
]}
],
 "PostToolUse": [
 { "matcher": "Edit|Write", "hooks": [
 { "type": "command", "command": ".claude/hooks/post-edit-format.sh" },
 { "type": "command", "command": ".claude/hooks/activity-log.sh", "async": true }
]},
 { "matcher": "Bash", "hooks": [
 { "type": "command", "command": ".claude/hooks/activity-log.sh", "async": true }
]}
],
 "Stop": [
 { "matcher": "*", "hooks": [
 { "type": "command", "command": ".claude/hooks/coach-suggest.sh" }
]}
]
}
}

```

## C. Hook activity-log (PostToolUse async)

```

#!/usr/bin/env bash .CLAUDE/HOOKS/ACTIVITY-LOG.SH
Log léger zéro-token des Write/Edit/Bash dans .claude/logs/activity.log
set -e
cd "$(dirname "$0")/../../.."

INPUT="$(cat 2>/dev/null || echo '{}')"
TOOL=$(echo "$INPUT" | jq -r '.tool_name // empty' 2>/dev/null)
[-z "$TOOL"] && exit 0

LOG_DIR=".claude/logs"
LOG_FILE="$LOG_DIR/activity.log"
mkdir -p "$LOG_DIR"

TS=$(date '+%Y-%m-%d %H:%M:%S')
SID=$(echo "$INPUT" | jq -r '.session_id // empty' | cut -c1-8)
[-z "$SID"] && SID="-----"

case "$TOOL" in
 Bash)
 CMD=$(echo "$INPUT" | jq -r '.tool_input.command // empty' | tr '\n' ' ' | cut -c1-200)
 echo "[$TS] [$SID] BASH | $CMD" >> "$LOG_FILE" ;;
 Write)
 FP=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')
 echo "[$TS] [$SID] WRITE | $FP" >> "$LOG_FILE" ;;
 Edit)

```

```

 FP=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')
 echo "[\$TS] [\$SID] EDIT | \$FP" >> "$LOG_FILE" ;;
esac

Rotation : si > 5000 lignes, garder les 3000 dernières
LINES=$(wc -l < "$LOG_FILE" 2>/dev/null || echo 0)
if ["$LINES" -gt 5000]; then
 tail -3000 "$LOG_FILE" > "$LOG_FILE.tmp" && mv "$LOG_FILE.tmp" "$LOG_FILE"
fi

exit 0

```

## D. Hook pre-tool-guard (PreToolUse défensif)

Cf. chapitre 3 ci-dessus. Ne pas oublier `chmod +x` sur tous les hooks.

## E. Hook session-start (SessionStart récap)

Cf. chapitre 3 ci-dessus.

## F. Sub-agent `security-auditor`

```

name: security-auditor
description: >
 Audit READ-ONLY de sécurité. Use proactively quand l'utilisateur dit
 "audite la sécurité", "RGPD", "vérifie les fuites", ou avant un déploiement
 public. Couvre : auth, secrets, injections, XSS, exposition PII, OWASP Top 10,
 conformité RGPD basique. Ne modifie aucun fichier.
model: opus
tools: Read, Grep, Glob, Bash, WebFetch, WebSearch

Tu es un auditeur sécurité spécialisé apps SaaS.

Checklist (à dérouler systématiquement)

1. Auth & sessions
- Vérification user côté server avant accès aux ressources ?
- Server actions / routes API scopent l'user_id ?
- Tokens en cookies HttpOnly+Secure ? Refresh ?

2. Row-Level Security (si Postgres/Supabase)
- Pour chaque table sensible : policy "user only accesses own X" ?
- INSERT/UPDATE/DELETE policies présentes (pas juste SELECT) ?
- RLS activée sur TOUTES les tables sensibles (sinon = faille critique) ?

3. Injections / Validation
- Input user inséré tel quel en DB ?
- Validation côté serveur (Zod / Pydantic) ?
- File upload : mime/size limit, antivirus ?

4. XSS / CSRF

```

- `dangerouslySetInnerHTML` justifié ?
- Liens externes : `rel="noopener noreferrer"` ?

### 5. Secrets & exposition

- `.env\*` dans `.gitignore` ?
- Clés API hardcodées ? (`grep -r "sk-"`, `grep -r "API\_KEY"`)
- `NEXT\_PUBLIC\_\*` contient uniquement du public ?

### 6. RGPD (si users UE)

- Consentement collecté ? CGU / Privacy policy ?
- Droit à l'export ? À l'oubli ? Cascade delete ?

### 7. Headers & infra

- HSTS, CSP, X-Content-Type-Options ?

## Format de sortie

Pour CHAQUE finding :

- **Sévérité** :  /  /  / 
- **Fichier:ligne** ou endroit (ex: "table users")
- **Risque** : ce qui peut arriver concrètement
- **Fix recommandé** : code/SQL (ne pas l'appliquer)

Termine par : Score /10 + Top 3 actions urgentes.

## Règles

- Distinguer "bloquant pour public" vs "à faire avant scale".
- Si faille critique exploitable maintenant → la dire en premier.
- Lecture seule strict. Pas de fix automatique.

## G. Slash command `/ship` complet

Cf. chapitre 5 ci-dessus.

## Checklist "30 minutes pour démarrer"

ÉTAPE	ACTION	DURÉE
1	Installer Claude Code ( <code>curl -fsSL https://claude.ai/install.sh   bash</code> ou Homebrew, ou extension IDE)	2 min
2	Créer <code>CLAUDE.md</code> à la racine (template annexe A)	10 min
3	Créer <code>.claude/settings.json</code> avec permissions (annexe B partiel)	5 min
4	Mettre à jour <code>.gitignore</code> (whitelist <code>.claude/</code> )	2 min
5	Tester : poser à Claude une question sur la stack du projet	3 min
6	Optionnel : créer hook PreToolUse défensif (annexe D)	8 min
<b>Total</b>	<b>≈ 30 minutes pour un Claude transformé</b>	

---

*Le Code du Claudeur — Volume 1. Document vivant : la V2 ajoutera "plugins partageables en équipe" et "intégration CI/CD". Feedbacks bienvenus. Rédigé pour qu'on arrête de perdre du temps à réexpliquer à Claude ce qu'on lui a déjà dit hier.*